



TesseraApp

SYSTEM ARCHITECTURE & USER INTERFACE DESIGN · V3.0

TesseraApp Architecture & UI Design

A full-stack Customer Identity & Access Management system on
Angular 21 and Spring Boot 4.

AUTHORS

Robert C. Oliver, Jr. · Travis L. Lester

INSTITUTION

Lewis University

COURSE

CPSC 69100 — Master's Project

DATE · VERSION

July 2026 · v3.0

Overview

This document presents the architecture and user-interface design of **TesseraApp**, a zero-trust CIAM platform we built on Angular 21, Spring Boot 4, and MySQL 8.

We describe our architecture through four views — logical, process, development, and physical — followed by the graphical user-interface design and screen prototypes. The system implements a zero-trust authentication core (stateless JWT access tokens with server-tracked, rotating refresh sessions), permission-based role access control, multi-factor authentication, and federated login.

4+1

ARCHITECTURAL VIEWS

7

RBAC ROLES

JWT

HMAC-SHA512 TOKENS

Table of contents

1	Introduction	03
2	Architectural goals & constraints	03
3	Logical view	04
4	Process view	06
5	Development view	09
6	Physical view	10
7	User-interface design	11
8	Design rationale & trade-offs	15
9	UML diagrams	16
10	References	18

01 Introduction

1.1 Purpose

This document specifies the software architecture and UI design of TesseraApp. It bridges the Software Requirements Specifications (SRS) and our implementation which is a structural and behavioral account of what we built and how it can be extended.

1.2 Scope

TesseraApp is a zero-trust CIAM platform we built covering: secure registration and login, multi-factor authentication (authenticator-app TOTP and SMS), federated sign-in (OAuth2/OIDC), role-based access control across seven roles*, organization-scoped administration, full audit logging, and a business domain of customer and invoice management with report export.

DRIVERS

02 Architectural goals & constraints

GOAL	ARCHITECTURAL RESPONSE
Zero-trust, scalable auth	Stateless JWT access tokens, verified by signature with no database hit per request
Revocable sessions	Server-tracked refresh sessions with rotation + reuse detection
Least privilege	Permission-string RBAC enforced at URL <i>and</i> method level
Auditability	Event-driven audit trail of every security-relevant action
Separation of concerns	Layered backend; standalone-component frontend

KEY CONSTRAINTS

We built on Java 21 / Spring Boot 4 / Spring Security 7, Angular 21, MySQL 8, and HMAC-SHA512-signed JWTs. The frontend currently targets a fixed API origin (<http://localhost:8080>) — a known gap we document in §8.

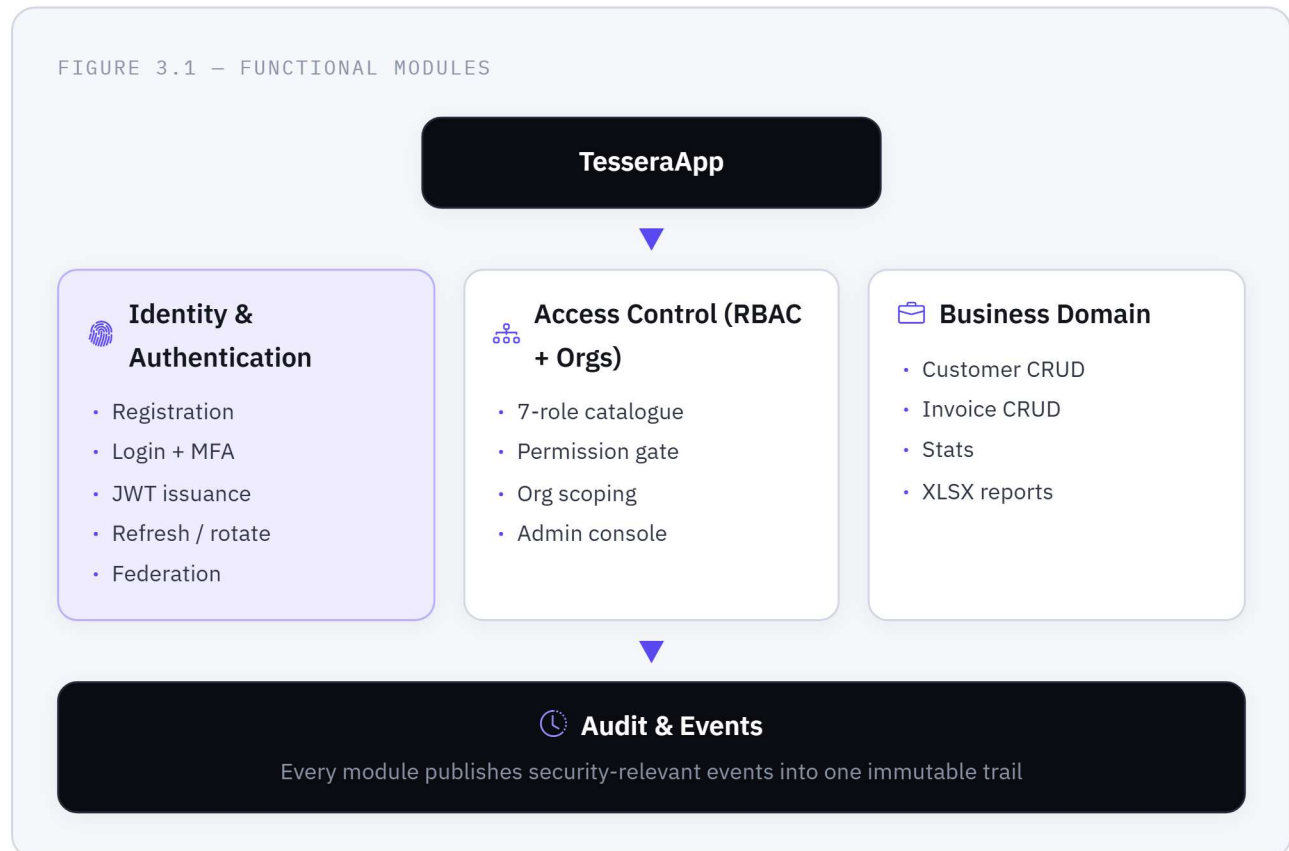
AI TOOL DISCLOSURE

AI tools were used to assist with the preparation of this and other course deliverables throughout this project. All design decisions, architectural analysis, and academic content reflect our own understanding and direction. We take full responsibility for the accuracy of all submitted work.

03 Logical view

This section covers what TesseraApp does and how we broke it into cooperating functional modules.

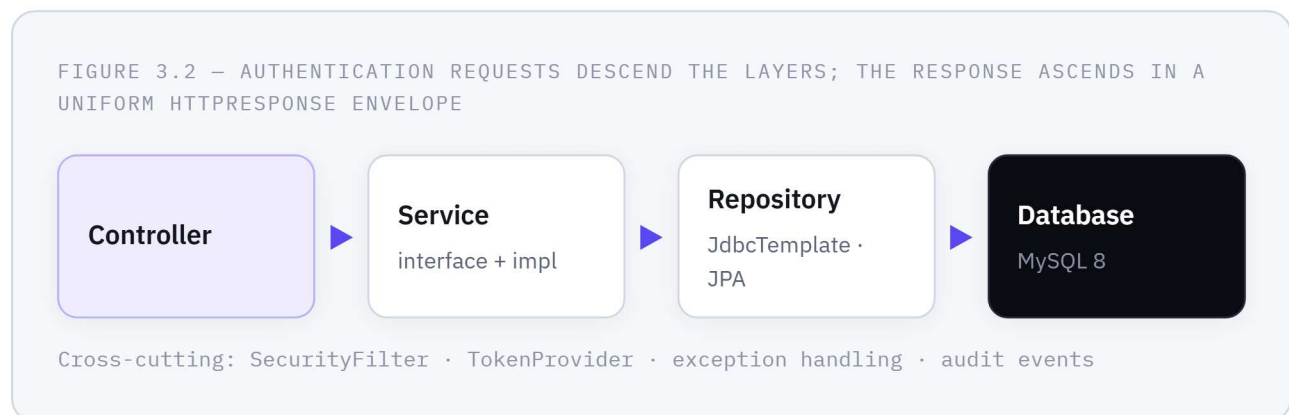
3.1 Functional decomposition



3.2 Key abstractions (domain model)

ABSTRACTION	ROLE
User	The account (profile + auth flags); Java object mapped from <code>users</code>
Role + permission	One role per user; permissions are <code>RESOURCE:ACTION</code> grants
UserPrincipal	Spring Security <code>UserDetails</code> adapter exposing authorities
RefreshSession	One device login — a <i>family</i> of rotating refresh tokens
TotpCredential / MfaChallenge	Authenticator secret + first-factor proof
OAuthProviderLink	Binding of a local user to an external identity
Organization / membership	Scoping unit for organization admins
UserEvent	An audit-log entry
Customer / Invoice	Business entities (JPA)

3.3 Layered structure



04 Process view

This section covers how TesseractApp behaves at runtime — the threading model, concurrency decisions, and the sequences we considered most critical.

4.1 Concurrency model

- **Stateless request handling.** Each HTTP request is served on a Tomcat worker thread with no server-side session; the JWT carries all state, making the API horizontally scalable.
- **Per-request security context.** CustomAuthFilter populates a thread-local SecurityContext for the duration of the request only.
- **Frontend refresh concurrency.** Concurrent 401s in the SPA are funnelled through a single token refresh (an RxJS BehaviorSubject guard) to avoid a thundering herd.

4.2 Sequence — login with token issuance

Angular SPA	UserController	AuthManager	SessionService	MySQL
1	Submit credentials POST /user/login Angular → UserController			
2	Brute-force gate , then authenticate UserController → AuthManager			
3	BCrypt verify → returns UserPrincipal AuthManager → MySQL → AuthManager			
4	Evaluate second factor (TOTP / SMS), then issueTokenPair() and insert session UserController → SessionService → MySQL			
5	Return { access, refresh }; SPA stores tokens and is ready UserController → Angular			

4.3 Sequence — refresh with reuse detection

Angular SPA	API	SessionService.rotate()	MySQL
1	A call returns 401 ; SPA requests a refresh <code>GET /refresh</code> Angular → API		
2	Verify JWT + passwordChangedAt, then look up the token's jti API → SessionService → MySQL		
3	If superseded / revoked → revoke the whole family, audit TOKEN_REUSE, and throw SessionService → MySQL (commit)		
4	Else → supersede current token and insert a new one SessionService → MySQL		
5	Return rotated { new tokens } API → Angular		

4.4 Sequence — registration & email verification

Angular SPA	UserController	UserService	EmailService	MySQL
1	Submit registration <code>POST /user/register</code> (public) Angular → UserController			
2	Check email uniqueness, insert user (enabled = false), assign ROLE_USER UserController → UserService → MySQL			
3	Store a UUID verification key, then dispatch the verification email UserService → EmailService			
4	Return 201 Created; SPA shows “check your inbox” UserController → Angular			
5	User clicks the emailed link <code>GET /user/verify/account/{key}</code> → enabled = true; login now permitted Angular → UserController → MySQL			

4.5 Sequence — federated login (OAuth2 / OIDC)

Angular SPA	Spring OAuth2	Provider (IdP)	OAuth2SuccessHandler	SessionService
1	Full-page redirect (not XHR) <code>/oauth2/authorization/{provider}</code> Angular → Spring OAuth2			
2	302 to provider consent; user approves Spring OAuth2 → Provider			
3	Callback <code>/login/oauth2/code/{provider}</code> ; Spring validates state, exchanges code, verifies id_token Provider → Spring OAuth2			
4	findOrCreateFederatedUser keyed on (provider, subject) OAuth2SuccessHandler → FederatedIdentityService			
5	If MFA → redirect to challenge; else issueTokenPair() and redirect to <code>/oauth2/callback#</code> with tokens OAuth2SuccessHandler → SessionService → Angular			

STATELESS CAVEAT (CON-3)

Federation is the one exception to the stateless model: the OAuth2 handshake briefly uses the container session to hold the CSRF state parameter between the outbound redirect and the callback. No SecurityContext is ever persisted — the session plays no part after token issuance.

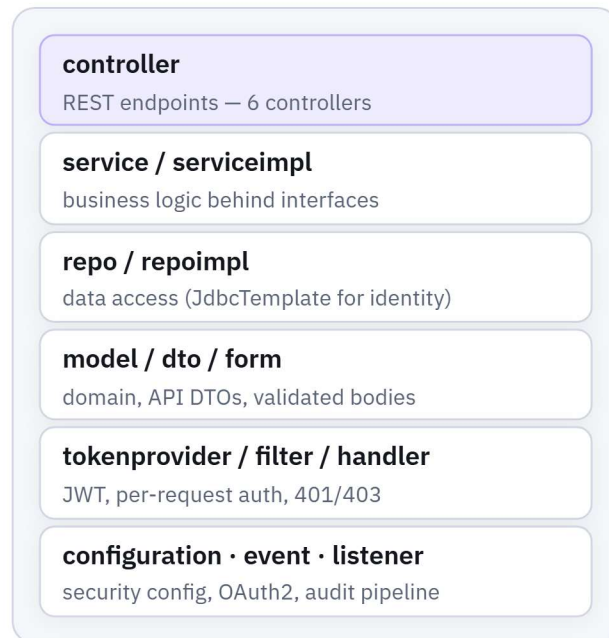
4.6 AUDIT PROCESS

Controllers publish a `NewUserEvent`; a listener writes the `userevents` row via the event publisher, keeping audit writes off the main logic path.

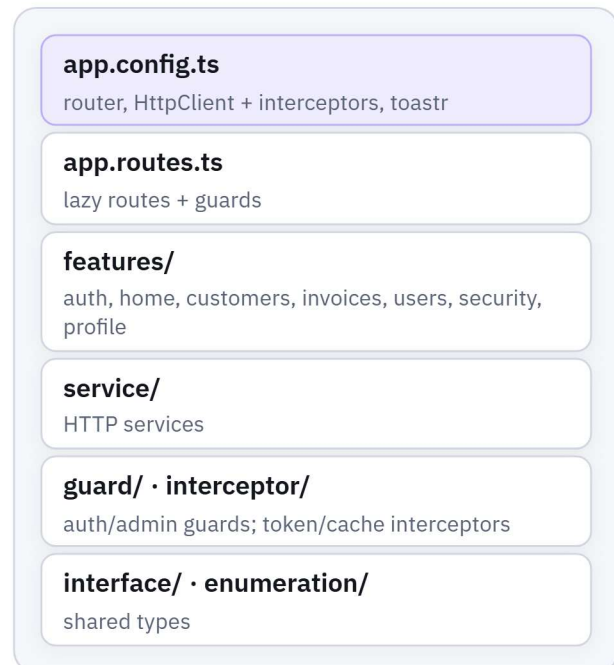
05 Development view

This section covers how we organized the codebase — package structure, module boundaries, and the build pipeline.

5.1 Backend (layered packages)



5.2 Frontend (standalone Angular)



5.3 Build & dependency structure

LAYER	TOOLCHAIN & KEY LIBRARIES
Backend	Maven (Spring Boot 4, Java 21); Spring Security 7, auth0/java-jwt, ZXing (TOTP QR), Apache POI (XLSX), Twilio, OWASP dependency-check
Frontend	npm / Angular CLI 21; Bootstrap 5, RxJS, ngx-toastr, jsPDF
Packaging	Three-stage Docker build: compile Angular → bake into the Spring Boot JAR → run on a slim JRE

5.4 LAYERING RULES

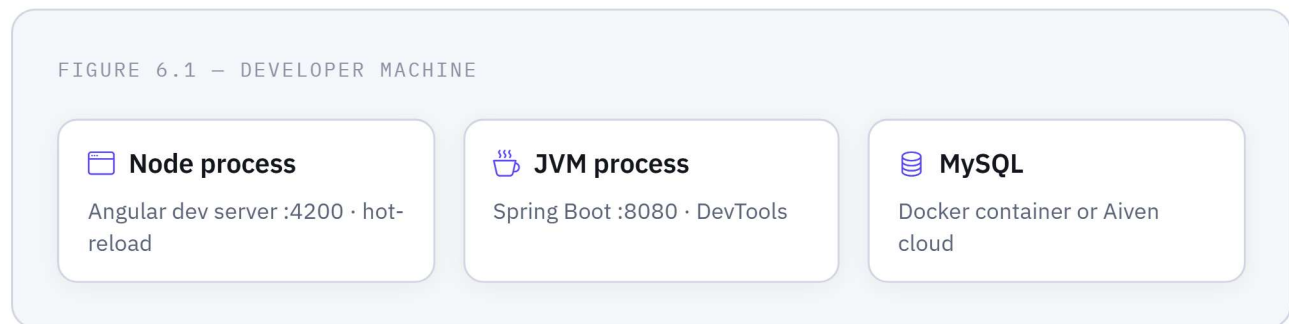
Dependencies point downward only (controller → service → repo). The identity layer never depends on the business layer; cross-cutting concerns (security, audit) are isolated in their own packages.

06 Physical view

This section covers where everything runs — from our dev machines to the containerized production topology.

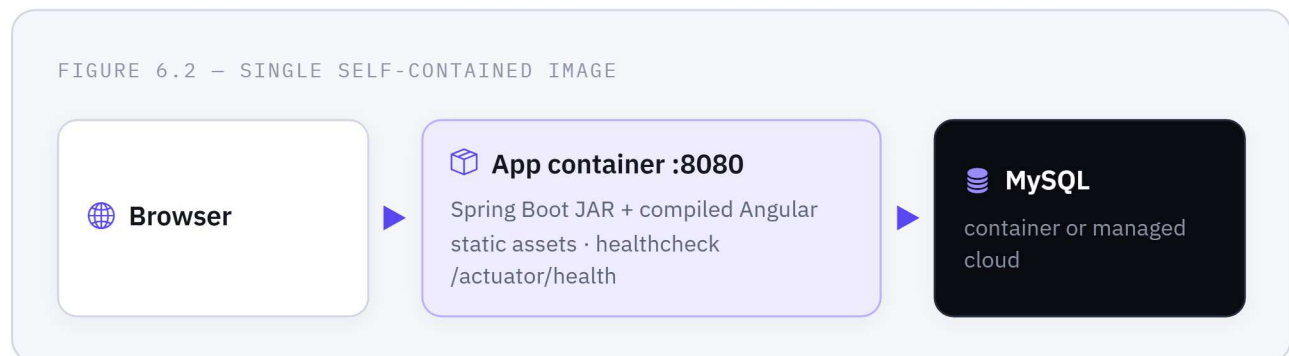
6.1 Development topology

FIGURE 6.1 — DEVELOPER MACHINE



6.2 Containerized / production topology

FIGURE 6.2 — SINGLE SELF-CONTAINED IMAGE



07 User-interface design

7.1 Design system

TesseraApp uses a dark-first "security console" visual language — deep space-blue surfaces, a single electric-iris accent, frosted-glass cards, and IBM Plex Sans/Mono. The token layer themes Bootstrap 5 and supports a full light mode from one data-bs-theme attribute.

- **Feedback:** non-blocking toasts for success/error; explicit LOADING / LOADED / ERROR states via a DataState enum so every async view has a defined visual.
- **Navigation:** a persistent authenticated navbar (profile, security, admin links shown by role); public auth pages render without it.
- **Consistency:** uniform forms, validation messaging, and a single error-presentation pattern fed by the API's HttpResponseMessage envelope.

7.2 Authentication — login & MFA

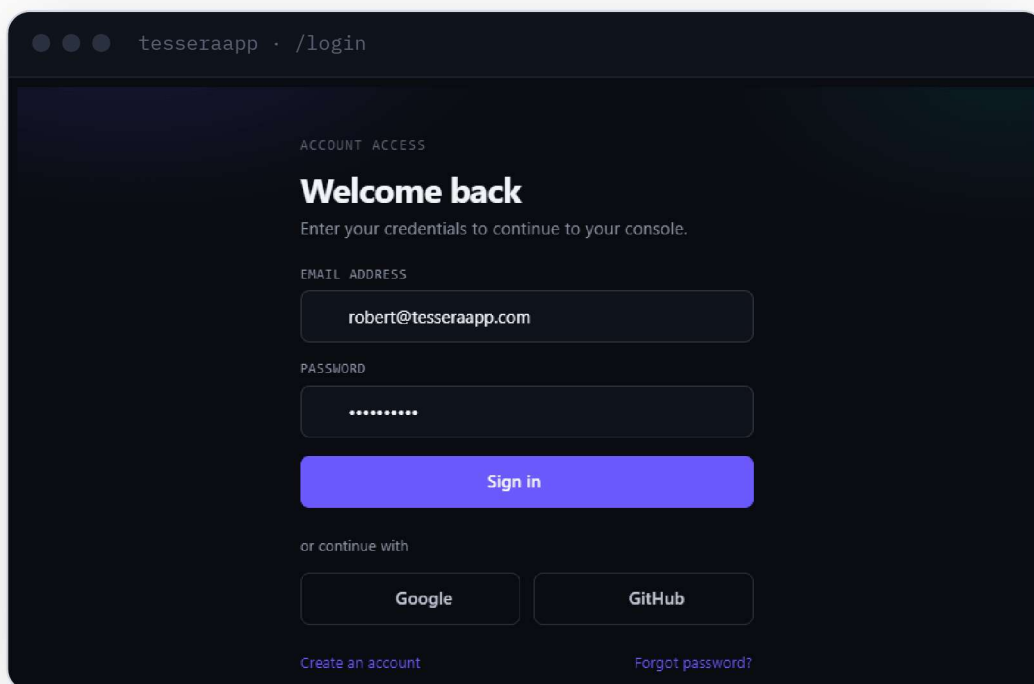


FIGURE 7.1 The access console. The credential form branches into a 6-digit MFA challenge (authenticator or SMS) when a second factor is enabled.

7.3 Dashboard & business domain

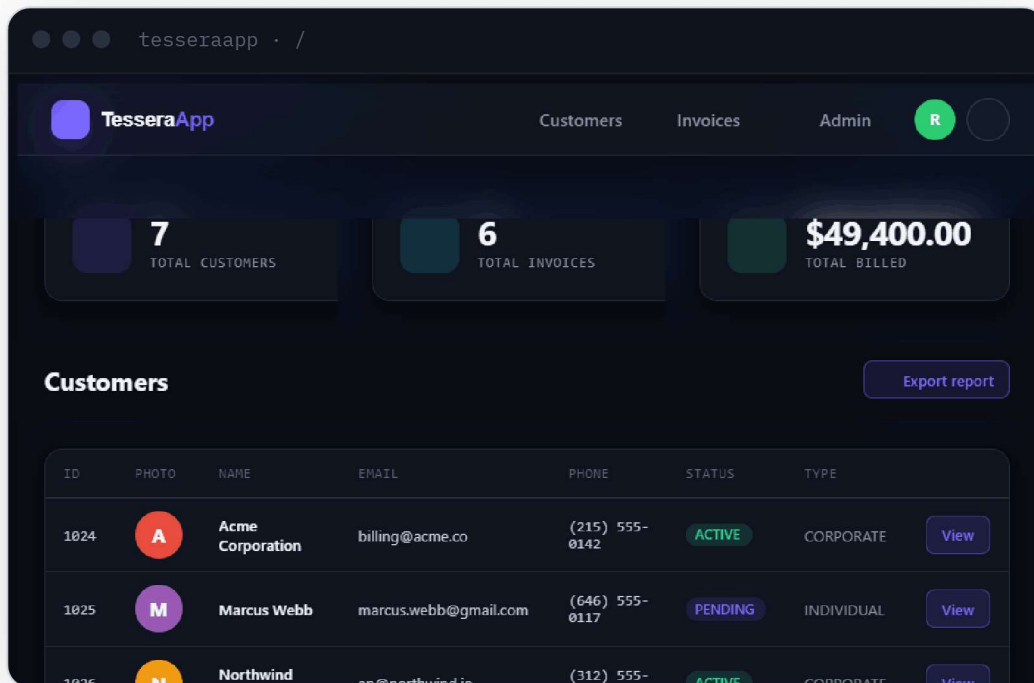


FIGURE 7.2 The home dashboard — KPI stat tiles (customers, invoices, total billed) above a paginated, searchable customers table with status chips.

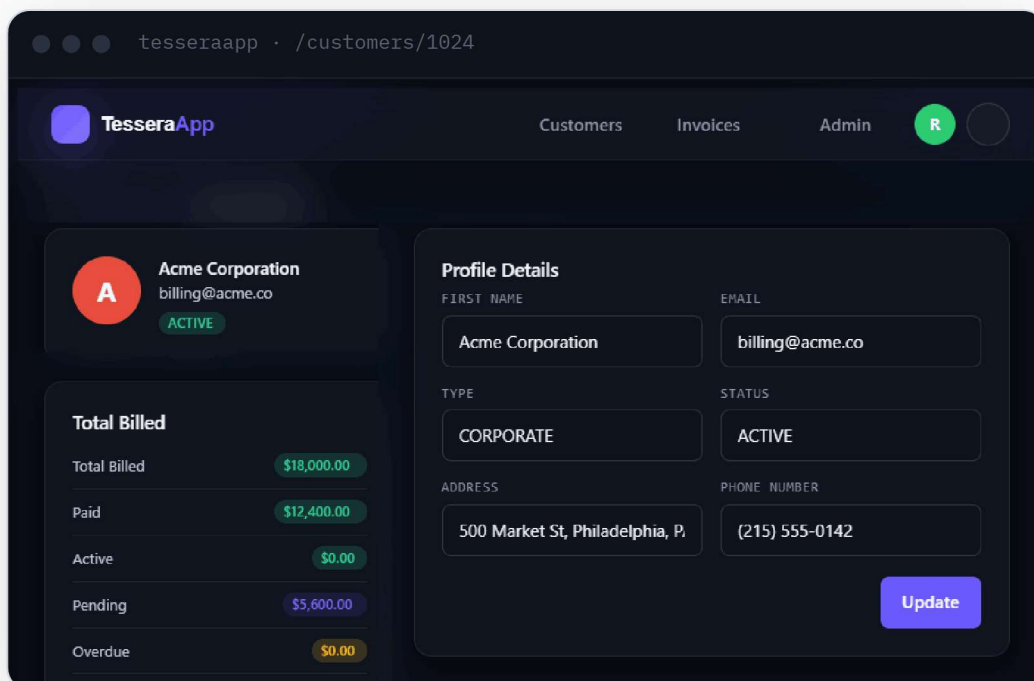


FIGURE 7.3 Customer detail — a billed breakdown by invoice status beside an editable profile form, gated by the viewer's role.

7.4 Invoicing & security self-service

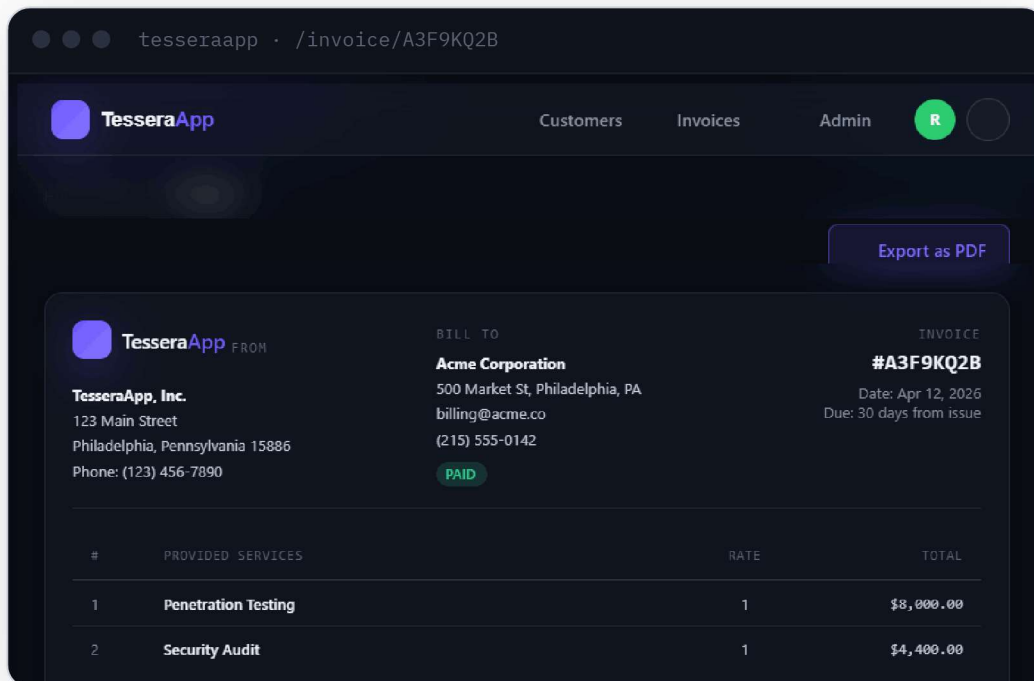


FIGURE 7.4 The printable invoice — itemised services, subtotal, tax and total, with PDF export.

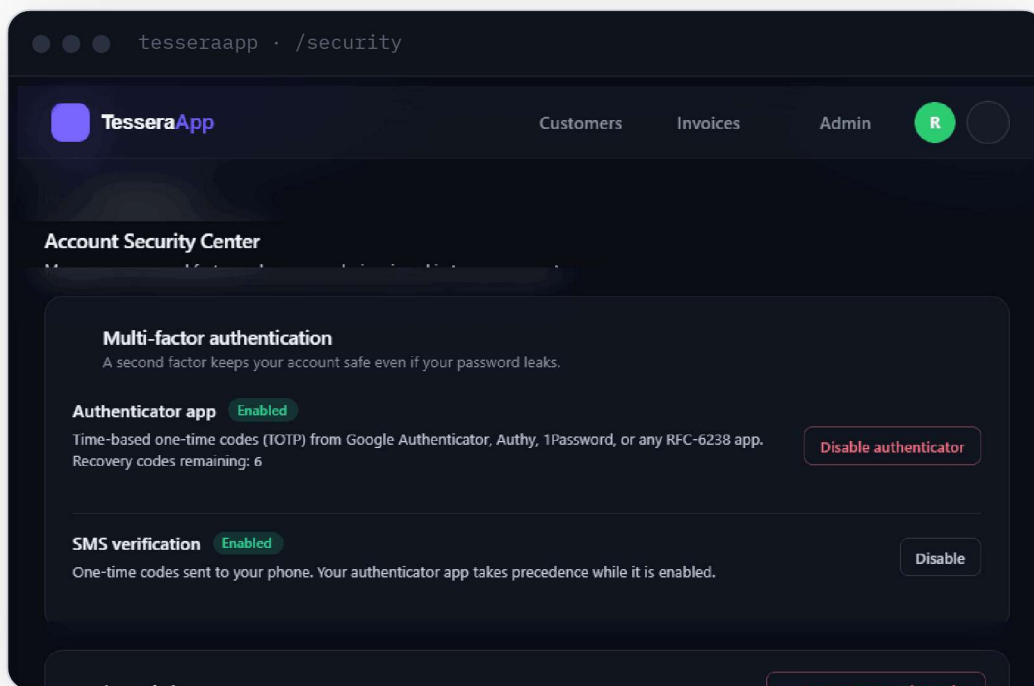


FIGURE 7.5 The Account Security Center — authenticator enrollment, the live device/session list, and the per-account activity history (UC-3, UC-5).

7.5 Administration

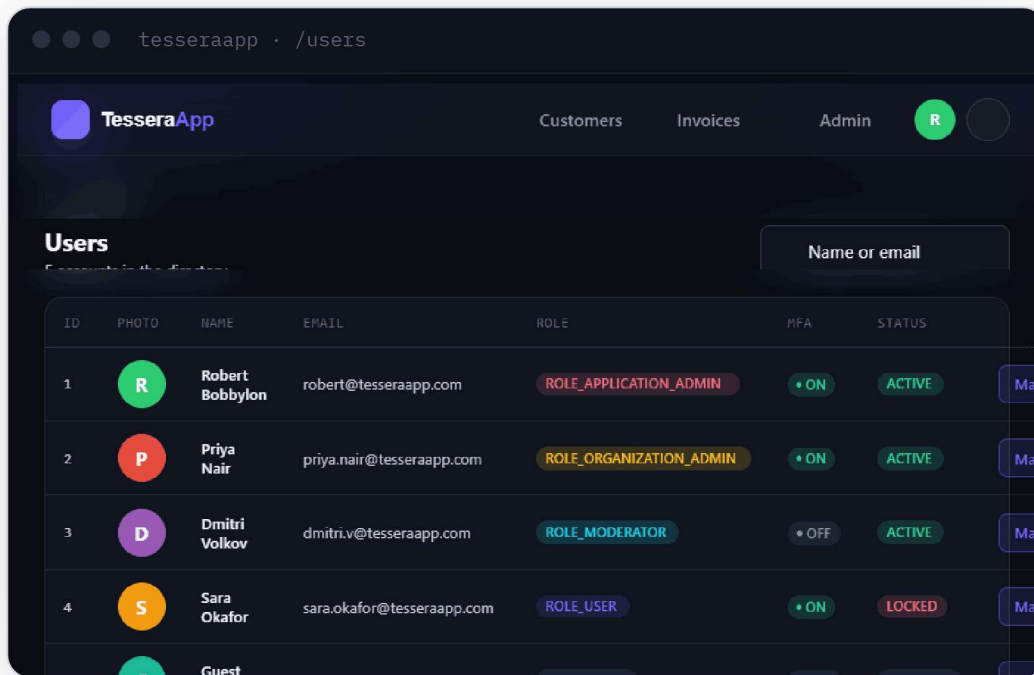


FIGURE 7.6 The admin user directory — searchable, with role, MFA and account-state chips, org-scoped for organization admins.

7.6 Primary user flows

- **Authentication:** Login → (MFA) → Dashboard; Register → email verify → Login; Forgot password → email → reset.
- **Self-service security:** Dashboard → Security Center → enroll authenticator / revoke a session.
- **Administration:** Dashboard → Users → select user → change role / toggle state (audited against the target).
- **Business:** Customers → New customer; Invoices → New invoice → attach to customer → export report.

08 Design rationale & trade-offs

DECISION	RATIONALE	TRADE-OFF
Stateless access + stateful refresh	Scalable requests <i>and</i> revocable sessions	Two token types to manage
JdbcTemplate (identity) + JPA (business)	Explicit, auditable SQL where security lives	Two persistence idioms
Permission-string RBAC, one role/user	Simple, readable least-privilege model	No multi-role users without change
Event-driven audit	Logging off the hot path	Indirection when tracing writes
Frontend guards as UX, backend as boundary	Defense in depth	Rules expressed in two places

CROSS-REFERENCE

This design is realised in code; the developer-facing companions are [documentation/architecture.md](#), [security.md](#), [database.md](#), and [api-reference.md](#).

09 UML diagrams

This section restates the four architectural views in standard UML notation, supplementing the descriptive diagrams in §§3–6.

9.1 Class diagram — logical view

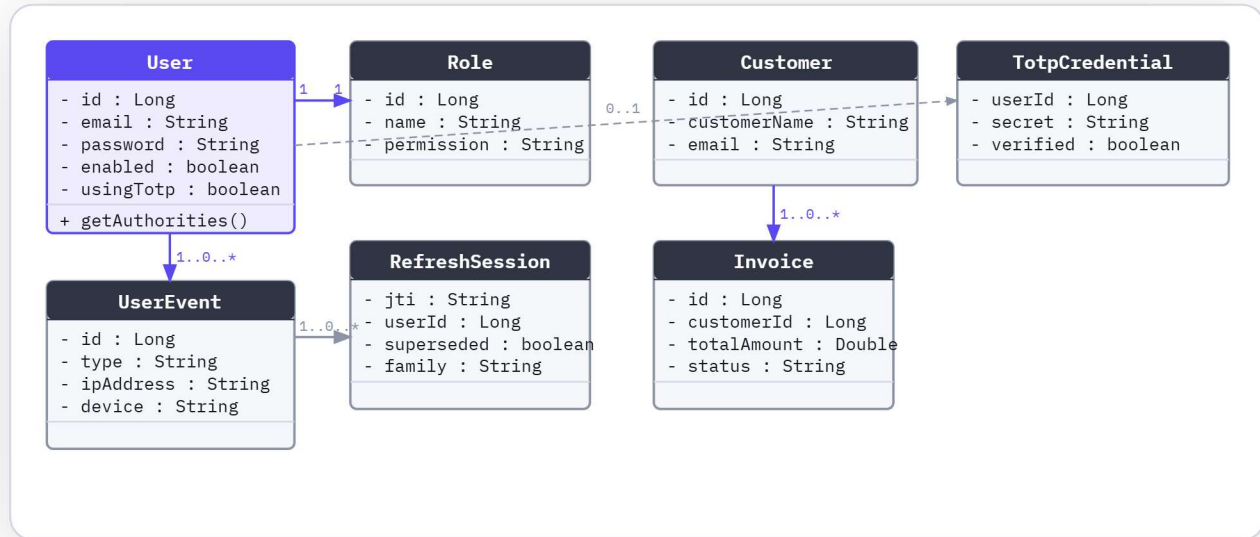


FIGURE 9.1 UML class diagram — core domain entities and associations. Multiplicity shown at association ends.

9.2 Sequence diagram — process view (login flow)

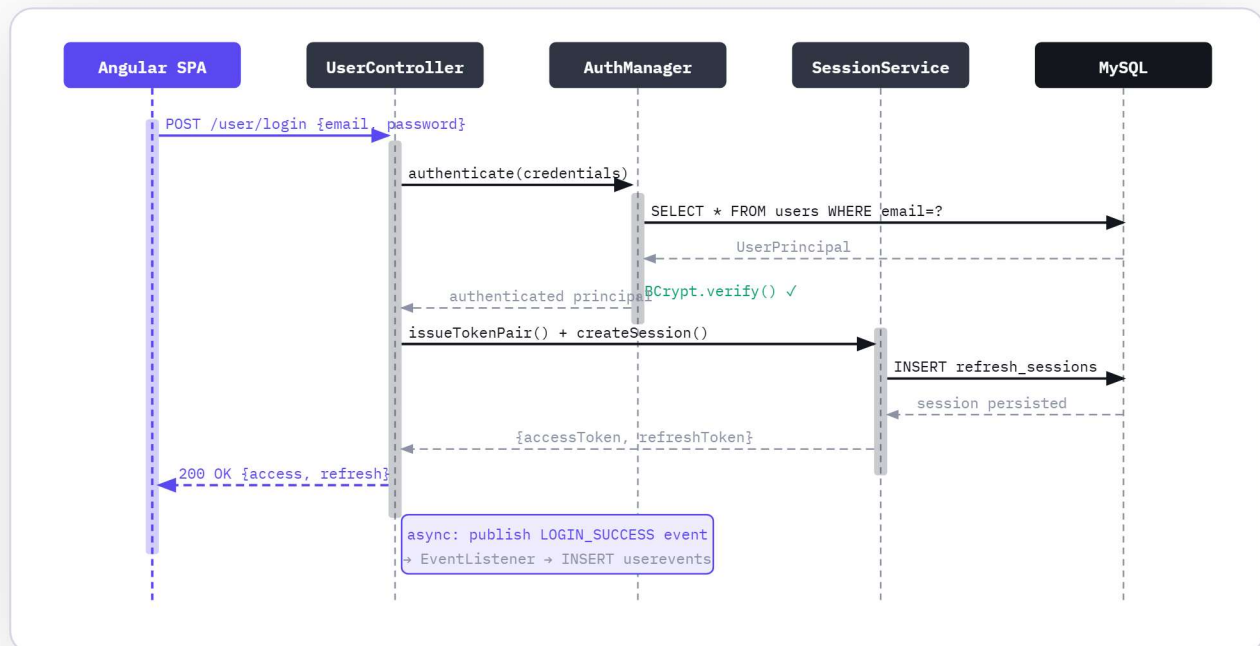


FIGURE 9.2 UML sequence diagram — login flow with activation bars. Dashed returns show responses; async audit note shows off-path event publishing.

9.3 Component diagram — development view

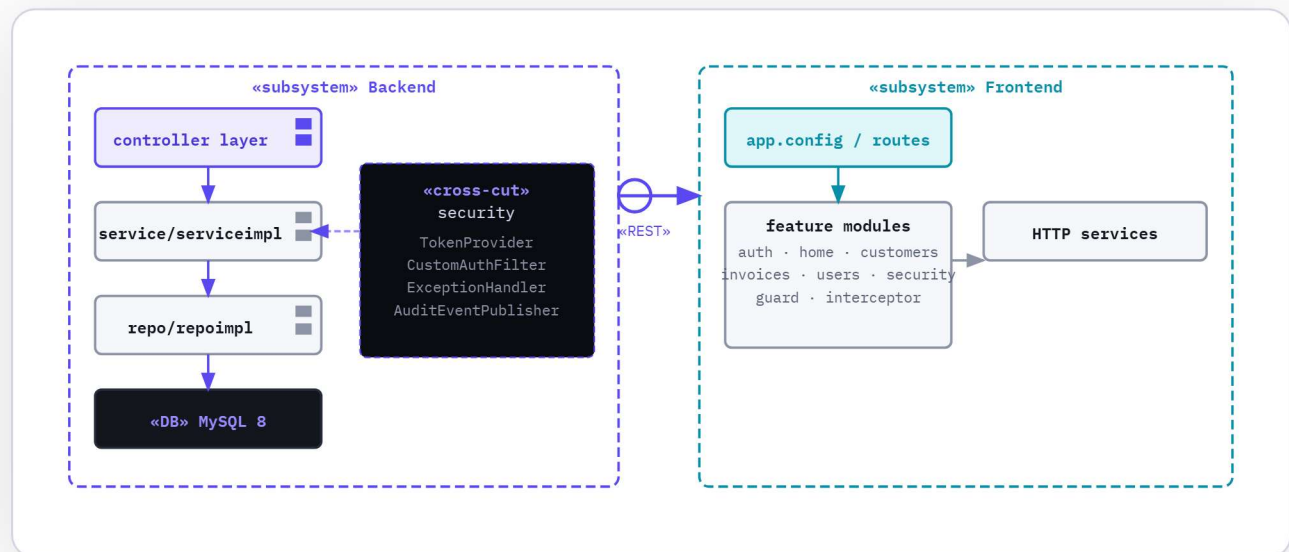


FIGURE 9.3 UML component diagram — backend and frontend subsystems. Lollipop connector at centre shows the REST API boundary.

9.4 Deployment diagram — physical view

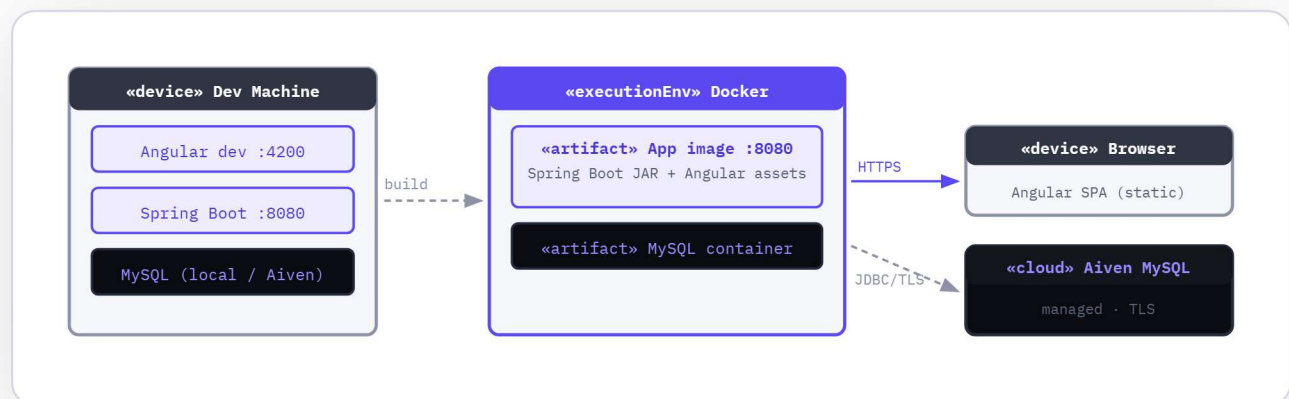


FIGURE 9.4 UML deployment diagram — developer machine, Docker container, browser client, and Aiven managed MySQL. Communication protocols on connectors.

10 References

The following sources informed our architectural approach, design decisions, and security implementation.

10.1 Architecture methodology

- [1] P. Kruchten, “Architectural Blueprints — The '4+1' View Model of Software Architecture,” *IEEE Software*, vol. 12, no. 6, pp. 42–50, Nov. 1995.
- [2] ISO/IEC/IEEE, “Systems and software engineering — Architecture description,” ISO/IEC/IEEE 42010:2011(E), Dec. 2011.
- [3] Wikipedia, “4+1 architectural view model,” retrieved July 2026. https://en.wikipedia.org/wiki/4+1_architectural_view_model

10.2 Security standards & guidance

- [4] NIST, “Digital Identity Guidelines — SP 800-63B,” rev. 4, 2024. <https://pages.nist.gov/800-63-4/sp800-63b.html>
- [5] OWASP, “Authentication Cheat Sheet,” OWASP Cheat Sheet Series, retrieved July 2026. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [6] IETF, “RFC 6238 — TOTP: Time-Based One-Time Password Algorithm,” May 2011. <https://datatracker.ietf.org/doc/html/rfc6238>
- [7] Auth0 by Okta, “JSON Web Tokens Introduction,” jwt.io, retrieved July 2026. <https://jwt.io/introduction>

10.3 Technology documentation

- [8] Spring Framework, “Spring Security Reference Documentation,” Spring.io, 2024. <https://docs.spring.io/spring-security/reference/>
- [9] Angular Team, “Angular 21 Documentation,” angular.dev, retrieved July 2026. <https://angular.dev/overview>
- [10] auth0/java-jwt, “Java JWT library,” GitHub, retrieved July 2026. <https://github.com/auth0/java-jwt>

10.4 Project source materials

- [11] bbobbylon, “angularSpringBootFullStack,” GitHub repository, 2024–2026. <https://github.com/bbobbylon/angularSpringBootFullStack>
- [12] Project documentation artifacts: SPRING_SECURITY_DETAILED_GUIDE.md, DOCUMENTATION_INDEX.md, api-reference.md, database.md — local repository, June–July 2026.
- [13] Aiven, “Aiven for MySQL — Managed cloud database service,” aiven.io, retrieved July 2026. <https://aiven.io/mysql>

AI Tool Disclosure

AI tools were used to assist with the preparation of this and other course deliverables throughout this project. All design decisions, architectural analysis, and academic content reflect our own understanding and direction. We take full responsibility for the accuracy of all submitted work.